

DA6400: Reinforcement Learning

Programming Assignment 2

Prof. Balaraman Ravindran

Kalyan S ed23b051 , Amritam H ee23b004 , Chandran K na23b040

[Drive link](#)

1 Vanilla DQN Implementation

The goal of DQN is to approximate the optimal action-value function:

$$Q^*(s, a) = \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (1)$$

The Bellman optimality equation is given by:

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a') \quad (2)$$

DQN approximates this using a neural network $Q(s, a; \theta)$ and minimizes the loss:

$$\mathcal{L}(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (3)$$

where θ^- are the parameters of the target network. We use the MountainCar-v0 environment with discrete actions. The truncation limit is modified to 2000 steps to allow sufficient time for learning momentum-based strategies. The discount factor is set to $\gamma = 0.99$.

1.1 Neural Network Architecture

The Q-network consists of:

- Input: 2-dimensional state (position, velocity)
- Two hidden layers of 16 neurons each with ReLU activation
- Output: Q-values for 3 discrete actions

Kaiming initialization is used to ensure stable training for ReLU activations.

1.2 Training Details

The following standard DQN components are used:

- ϵ -greedy exploration with decay
- Fixed-size replay buffer
- Uniform random sampling from buffer
- Hard target network updates every fixed steps
- Adam optimizer

The agent is trained for 200000 timesteps and evaluated across 15 random seeds.

```
GAMMA = 0.99
LR = 1e-3
BUFFER_SIZE = 50000
BATCH_SIZE = 64
TARGET_UPDATE = 500 # steps
EPS_START = 1.0
EPS_END = 0.05
EPS_DECAY = 50000
TOTAL_STEPS = 200000
MAX_EPISODE_STEPS = 2000
```

Figure 1: Hyper Parameters of MountainCar-v0

2 Result with 2000 truncation step

Figure 2 shows the return vs episodes curve averaged over 15 seeds, along with confidence intervals.

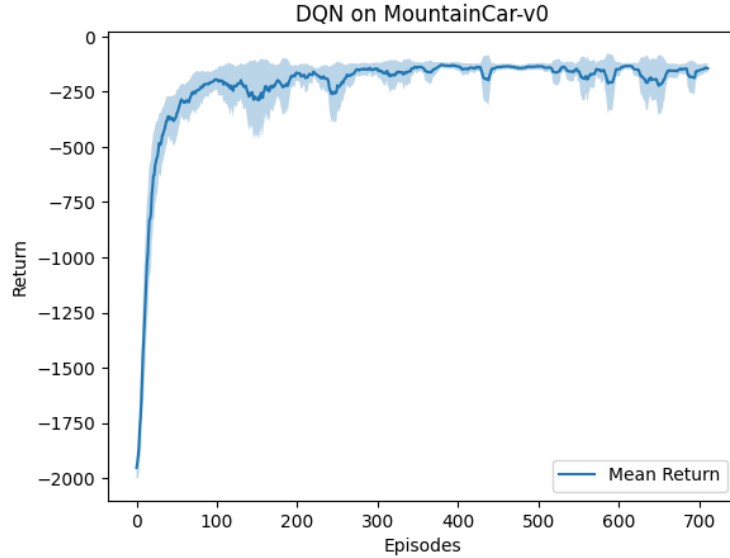


Figure 2: Return vs Episodes (With 95% Confidence Interval)

Initially, the return is very low (around -2000), indicating poor performance due to random exploration. As training progresses, the agent learns an effective strategy and the return improves steadily. Eventually, the performance stabilizes around -110 to -90, indicating near-optimal behavior.

The shaded region represents the confidence interval across 15 runs. The wide interval at the beginning indicates high variability due to exploration, while the narrowing interval later indicates convergence. The optimal behavior learned by the agent is non-intuitive. The car does not directly accelerate towards the goal. Instead, it first moves in the opposite direction (left) to gain momentum.

This behavior emerges because the engine is not powerful enough to climb the hill directly. The agent learns to:

1. Move left to increase potential energy
2. Swing back and forth to accumulate momentum
3. Use this momentum to reach the goal

This demonstrates that reinforcement learning can discover strategies that are not immediately obvious and require long-term planning.

3 Length of the Episode

3.1 Result

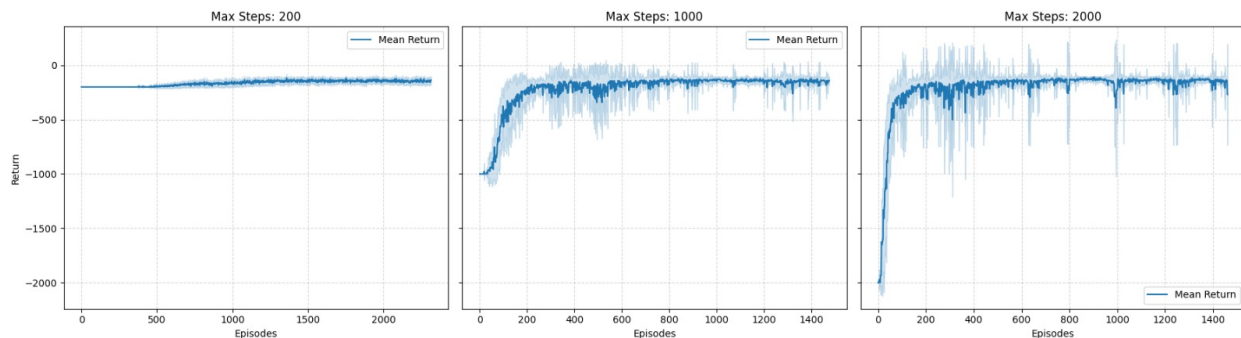


Figure 3: (Left) 200 steps , (Mid) 1000 steps , (Right) 2000 steps

3.2 Comparison of truncation lengths

- **200 steps:** Returns stay around ≈ -200 with an almost flat curve. Episodes end too early, so the agent rarely reaches the goal, leading to minimal learning and low variance.
- **1000 steps:** Learning improves quickly and stabilizes within ~ 100 – 200 episodes. Some variance exists (especially during early exploration), but overall performance is stable and meaningful.
- **2000 steps:** Similar mean performance as 1000 steps, but slower convergence and higher variance due to longer, noisier trajectories.
- **Observation:** Increasing truncation length helps learning, but beyond a point (e.g., 2000), it mainly adds noise without clear benefits.

3.3 Why truncation was introduced

In the original formulation, episodes were not artificially truncated. However, truncation was introduced in environments like Gymnasium for several practical reasons:

- **Prevent endless episodes:** A weak agent might never reach the goal, so truncation ensures training doesn't get stuck in a single episode.
- **Fair and consistent evaluation:** Fixed episode lengths make it easier to compare different models under the same conditions.
- **Better learning dynamics:** Regular resets help the agent see more diverse states and keep training efficient and stable.

3.4 Effects and appropriate truncation length

Drawbacks of truncation:

- **Skewed Value Estimates:** Treating a timeout as a terminal state forces a negative bias, as the agent ignores potential future rewards simply because the clock ran out.
- **Mixed Signals:** If the agent can't distinguish between a "hard stop" and an actual failure, the learning signal becomes confusing and leads to inconsistent policies.
- **Tuning Trade-offs:** Truncation adds another variable to manage; too short, and the agent never sees a win and if too long, and the data becomes too noisy to learn from efficiently.

Appropriate truncation length: The result shows that 200 steps is simply too short, cutting off the agent before it can actually figure out the task. On the other end, pushing it to 2000 steps just adds a lot of noise without really helping performance. A limit of around **1000 steps** seems to be the sweet spot as it gives the agent enough room to learn while keeping things stable.

4 Replay Factor Variants of DQN

4.1 Experiments for DQN with $\rho \in 1, 2, 4, 8$

4.1.1 Comparison of Results

Figures 4–4 show mean performance (10 seeds, 95% CI). All variants improve, but at different rates. $\rho = 4$ achieves the best performance (≈ -205) with the lowest variance, followed by $\rho = 2$. $\rho = 1$ learns slowest and is most variable, while $\rho = 8$ shows reduced stability despite reasonable performance. Overall ordering: $\rho = 4 > \rho = 2 > \rho = 8 > \rho = 1$.

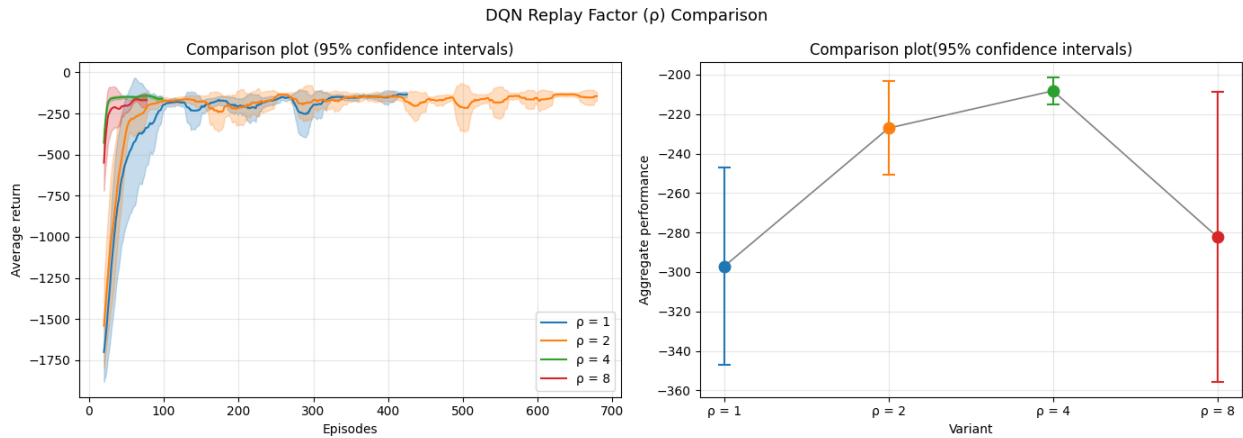


Figure 4: (Left) Learning curves. (Right) Aggregate performance (95% CI).

4.1.2 Statistical Significance

Differences are largely significant. $\rho = 1$ vs. $\rho = 4$ shows no CI overlap (clear difference). $\rho = 2$ vs. $\rho = 4$ partially overlaps (weaker evidence), while $\rho = 8$ overlaps with both, making comparisons inconclusive. Overall, $\rho = 1$ is clearly worse; $\rho = 4$ is strongest.

4.1.3 Effect of Increasing ρ

Higher ρ increases updates per step, improving **sample efficiency** and accelerating early learning. However, very high ρ (e.g. 8) leads to overfitting on stale replay data, causing instability and higher variance. Thus $\rho = 4$ balances fast learning and stability, achieving the best final performance within 200k steps.

4.1.4 When ρ Helps or Hurts

Advantage: When data collection is costly (e.g. robotics), higher ρ maximises learning from limited interactions.

Disadvantage: Large ρ increases compute and risks instability due to off-distribution updates, making it unsuitable for safety-critical or time-constrained settings.

4.1.5 Relation to Planning

With $\rho > 1$, replay acts as an **implicit model**, enabling multiple updates per real interaction, similar to Dyna-style planning. However, unlike true models, replay cannot generate new transitions—only reuse past ones—so this is best seen as **approximate planning** constrained by buffer quality.

4.2 Distribution of Performance

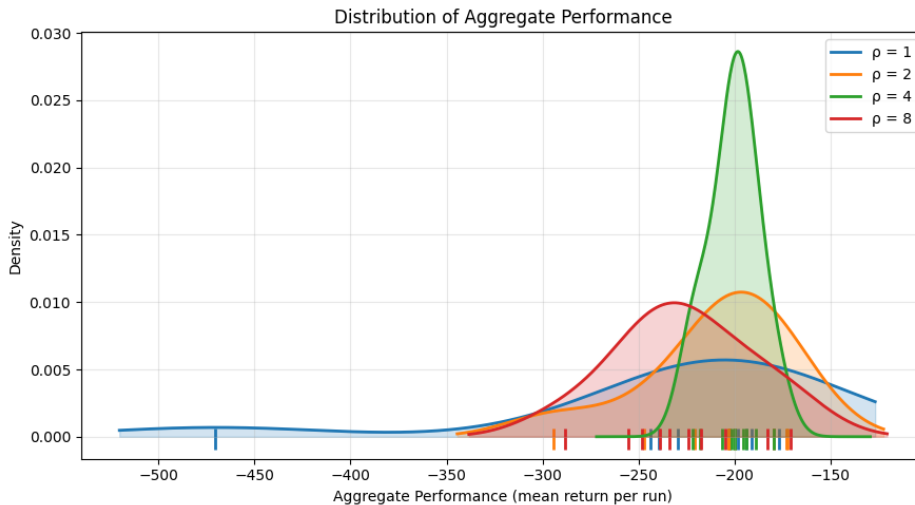


Figure 5: Distribution of Performance over various ρ

(i) Shape and Interpretation

The distributions are mostly **unimodal**, with $\rho = 4$ and $\rho = 2$ showing clear single peaks near higher returns. $\rho = 1$ appears **skewed** with a heavy tail toward poor performance, and may be weakly multimodal due to an outlier. $\rho = 8$ is broader and slightly skewed. None of the distributions are perfectly normal, though $\rho = 2$ and $\rho = 4$ are closer to symmetric. Intuitively, higher ρ (especially $\rho = 4$) leads to both better and more consistent performance, while $\rho = 1$ is unstable and unreliable.

(ii) Nuanced Inferences

Beyond mean performance, the distributions reveal differences in **variance and reliability**. $\rho = 4$ has the narrowest spread, indicating strong robustness across seeds. Although $\rho = 8$ achieves reasonable returns, its wider spread suggests instability. $\rho = 1$ shows high variance and susceptibility to poor outcomes. Thus, $\rho = 4$ is not only optimal in expectation but also in consistency.

(iii) Importance of Performance Distributions

Performance distributions are particularly useful in settings with high stochasticity, such as deep RL, hyperparameter tuning, or randomized optimization. Unlike learning curves, they expose **variance, skewness, and failure modes**, helping identify methods that are unreliable despite good average performance. This is critical in safety-sensitive or deployment scenarios where consistency matters as much as mean performance.

4.3 Variability in Performance (Tolerance Intervals)

4.3.1 Tolerance Interval Analysis



Figure 6: Variability in Performance ($\alpha = 0.05, \beta = 0.9$)

While the mean curves for all ρ values eventually reach a similar plateau, the intervals reveal a huge difference in reliability.

- **Instability at $\rho = 1, 2$:** The tolerance bands remain very wide throughout training. Even after hundreds of episodes, the lower bound stays close to -3000 . This shows that a large number of runs still end in total failure.
- **Convergence at $\rho = 4, 8$:** These setups show a noticeable narrowing of the intervals after about 40 episodes. This suggests that higher replay factors create much more consistent agents, no matter the initial seed.
- **Dont trust means bro:** Relying on the mean alone can be misleading. For lower ρ , the mean suggests success, but the TI indicates that the "worst-case" 10% of runs are still failing badly.

4.3.2 Reliability and Worst-Case Performance

The lower bound of the TI is an important measure for "worst-case" performance in real-world deployment. For $\rho = 1$ and 2, the agent is unreliable; its performance is very sensitive to initial parameters, making it a risk.

In contrast, $\rho = 8$ offers the best stability, with the lower bound stabilizing around -300 . This means that even in a bad run under $\rho = 8$, the agent will still work, while $\rho = 1$ may lead to an agent that cannot complete the task at all.

4.3.3 Tolerance vs. Confidence Intervals

Feature	Confidence Interval (CI)	Tolerance Interval (TI)
Focus	Estimates the uncertainty of a population parameter (e.g., the mean).	Estimates the range containing a specific proportion of individuals .
What it asks	"How sure are we about the average performance?"	"What is the range of results for my next single run?"
Use Case	Benchmarking and comparing the general superiority of algorithms.	Assessing reliability and safety-critical "worst-case" scenarios.
Limitations	Misleading in multimodal distributions (hides total failures in the average).	Requires larger sample sizes; sensitive to distributional assumptions.
As $n \rightarrow \infty$	Collapses to a point (the true population mean).	Converges to a fixed band (the true population spread).

4.4 Sensitivity Analysis

4.4.1 Mini-Batch Size

When looking at batch sizes, the two variants behave very differently.

- **For $\rho = 1$:** The performance is quite sensitive here. It starts off poor with smaller batches (scoring around ≈ -370 at a batch size of 16) but improves dramatically as the batch size grows, leveling off near ≈ -210 once we hit 64 and beyond.
- **For $\rho = 4$:** In contrast, this setting is incredibly stable. Regardless of the batch size we tested, the scores hovered steadily between ≈ -210 and ≈ -190 , showing that it doesn't really need massive batches to perform well.

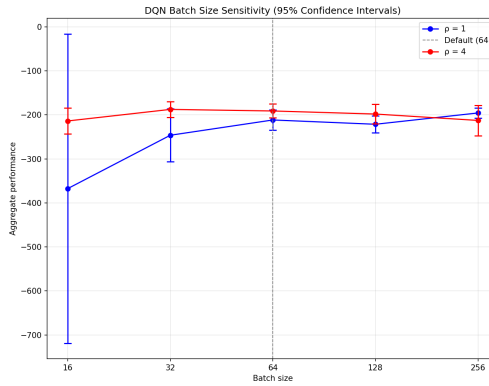


Figure 7: Sensitivity Analysis for Batch Size

4.4.2 Target Network Refresh Rate

Both configurations are surprisingly sensitive to how often the target network is updated, though they share a similar sweet spot.

- **For $\rho = 1$:** It performs reliably when the update interval is kept between 100 and 1000 steps. However, if we wait too long to refresh (e.g., 2000 steps), the targets become stale and performance tanks to roughly ≈ -420 . Updating too rapidly (at 100 steps) also introduces a bit of unwanted variance.
- **For $\rho = 4$:** It shares that same optimal window and is actually a bit more stable in the 250 to 1000 range. But just like the first variant, it suffers heavily from delayed updates, dropping even further to ≈ -560 at the 2000-step mark.

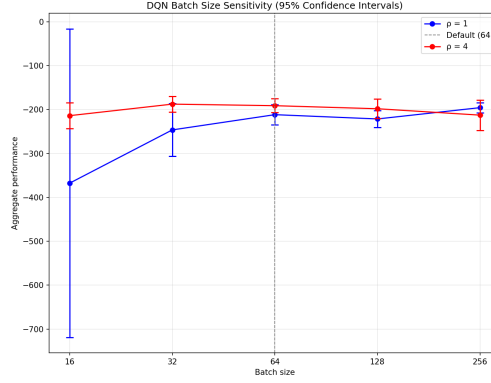


Figure 8: Sensitivity Analysis for Target Update

4.4.3 Deployment and Robustness

In deployment, avoiding failure is more important than achieving the best-case return.

- **Stable configuration:** The results support using $\rho = 4$ with moderate batch sizes $[32, 128]$ and target update interval $[250, 1000]$. This configuration consistently achieves returns around ≈ -200 with low variance. Large update intervals (e.g., **2000**) cause severe drop in performance (down to ≈ -560).
- **Risk with $\rho = 1$:** Although $\rho = 1$ can sometimes match performance, it is highly sensitive to hyperparameters, varying from ≈ -370 to ≈ -200 , making it unreliable.

5 Prioritized Experience Replay (PER)(BONUS)

The replay factor ρ determines how many gradient updates happen with each interaction in the environment. While uniform sampling treats all transitions the same, PER changes the sampling probability. It favors transitions with higher Temporal Difference (TD) error, which helps focus the learning budget on the most informative experiences.

- **Synergy with Replay Factor:** Increasing ρ gives more chances for optimization. Uniform sampling often wastes updates on "solved" transitions. PER, on the other hand, makes sure that as ρ increases, these extra updates focus on transitions that provide significant gradients. This makes every increase in ρ more effective.
- **Elevating the Performance Ceiling:** Uniform sampling reaches a point where higher ρ results in stagnation. PER raises this limit by ensuring the agent keeps learning from unexpected data. This improves sample efficiency as the computational budget grows.
- **Speed and Stability:** Agents using PER reach near-optimal values more quickly than those that use random sampling. It also serves as a stabilizer, leading to fewer deviations and tighter convergence once the agent gets close to its best performance.
- **Risk of Overfitting:** If ρ is increased too much ($\rho \gg 1$), the agent might keep sampling the same high-error transitions without updating them. This can cause local overfitting on a "surprising" part of the buffer that no longer matches the current needs of the value function.

Due to time constraints, we could only do 1 seed.

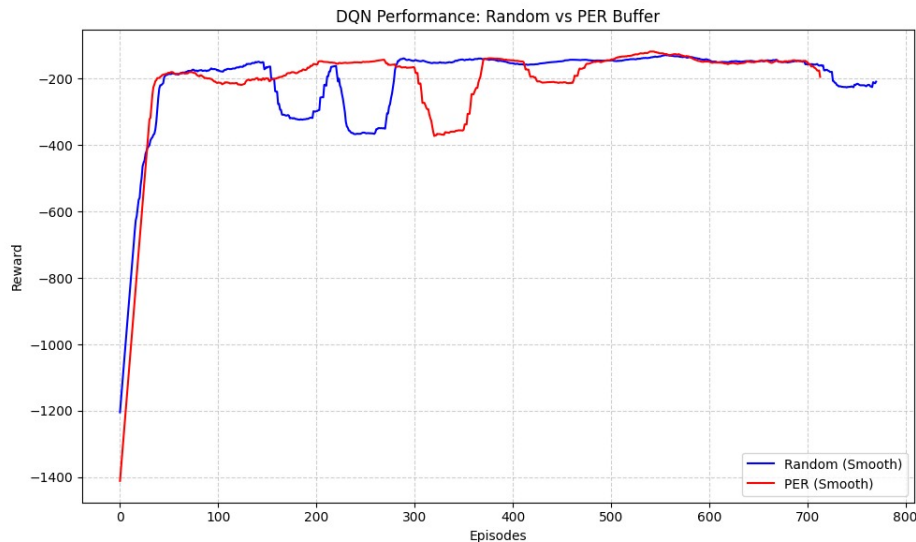


Figure 9: Random vs PER Buffer